TECH REFERENCE

Causal Expert & Optimal Transport Expert

기술 참조서: NOTEARS 인과 추론과 Sinkhorn 최적 수송

Author 1[1], Author 2[1]

[1]Organization Name
contact@org.com

범위(Scope).
- Causal Expert: Pearl SCM + NOTEARS 연속 비순환성 제약으로 피처 간 인과 DAG 학습
- OT Expert: Sinkhorn 엔트로피 정규화 최적 수송 → 고객↔프로토타입 Wasserstein 거리
- 공유 입력: 동일한 정규화 피처, 서로 다른 수학적 추출(인과 방향성 vs 분포 기하학)
- FP16 혼합 정밀도에서의 수치 안정성 이슈와 대응

설계 vs 구현 참고. 본 문서는 풀뱅크 설계(734D)를 기준으로 작성되었습니다. 현재 Santander 벤치마크 구현은 349D 입력 / Phase 0 후 403D (13 feature groups)입니다.

Contents

1 Causal Expert 4
1.1 Pearl 인과추론 배경 4
1.1.1 인과추론의 세 단계 4
1.1.2 상관관계의 함정과 교란 변수 4
1.2 NOTEARS 연속 DAG 제약 4
1.2.1 DAG 구조 학습의 난제 4
1.2.2 NOTEARS 핵심 아이디어 4
1.2.3 Taylor 10항 근사 5
1.3 SCM (Structural Causal Model) 개입 5
1.3.1 Feature Compressor 5
1.3.2 인과 개입 연산 5
1.3.3 Causal Encoder 6
1.4 DAG 정규화 손실 6
1.5 금융 도메인 적용 6
1.5.1 행동 간 인과 방향 학습 6
1.5.2 인과 그래프 추출 7
1.5.3 논문 원문 vs 구현 비교 7
2 Optimal Transport Expert 8
2.1 Wasserstein 거리 vs KL 발산 8
2.1.1 분포 비교의 기하학적 관점 8
2.1.2 Wasserstein 거리의 장점 8
2.2 Sinkhorn 엔트로피 정규화 8
2.2.1 Kantorovich 문제의 계산적 한계 8
2.2.2 엔트로피 정규화 9
2.2.3 Sinkhorn 알고리즘의 직관 9
2.3 Log-domain Sinkhorn 알고리즘 9
2.3.1 수치 안정성 문제 9
2.3.2 Log-domain 쌍대 변수 업데이트 9
2.4 학습 가능한 기준 분포와 PSD 비용 행렬 10
2.4.1 Distribution Projection 10

2.4.2 학습 가능한 기준 분포 ... 10
2.4.3 PSD 비용 행렬 ... 10
2.4.4 Wasserstein 거리 벡터 ... 10
2.4.5 Wasserstein Encoder ... 10
2.5 금융 도메인 적용 ... 10
2.5.1 세그먼트 간 분포 이동 ... 10
2.5.2 논문 원문 vs 구현 비교 ... 11
3 FP16 수치 안정성 ... 12
3.1 혼합 정밀도 (AMP) 환경의 도전 ... 12
3.2 Causal Expert FP16 이슈 ... 12
3.2.1 Taylor 근사에서의 오버플로우 ... 12
3.2.2 인접행렬 원소 폭주 ... 12
3.3 OT Expert FP16 이슈 ... 12
3.3.1 Sinkhorn log-domain에서의 문제 ... 12
3.4 Phase 2 NaN 수정 경험 ... 12
4 구현 참고사항 ... 14
4.1 두 Expert의 공통 설계 패턴 ... 14
4.2 PLE 통합 ... 14
4.3 세 Expert의 수학적 관점 비교 ... 14
4.4 Gradient Flow ... 15
4.4.1 Causal Expert ... 15
4.4.2 OT Expert ... 15
4.5 디버깅 가이드 ... 15
4.5.1 Causal Expert 이슈 ... 15
4.5.2 OT Expert 이슈 ... 15
4.6 추가 자료 ... 16

Causal Expert

Pearl 인과추론 배경

인과추론의 세 단계

Judea Pearl은 인과추론의 인식론적 수준을 세 단계로 구분하였다.

제1단계: 연관(Association). 관측 데이터에서 변수 간 상관관계를 파악한다. $P(Y \mid X)$ — "X를 관측했을 때 Y의 확률은 어떻게 변하는가." 전통적 추천 시스템은 이 수준에서 동작한다. "프리미엄 카드 보유자가 여행 보험 가입률이 높다"는 연관이며, 인과 방향을 구분하지 않는다.

제2단계: 개입(Intervention). 변수를 인위적으로 조작했을 때의 효과를 추정한다. $P(Y \mid \mathrm{do}(X = x))$ — "X를 x로 설정하면 Y는 어떻게 변하는가." 이 수준에서는 교란 변수(confounder)의 영향을 통제할 수 있다. 예: 소득 수준이라는 교란 변수를 통제한 상태에서, 프리미엄 카드 제공이 보험 가입에 미치는 순수 인과 효과를 추정한다.

제3단계: 반사실(Counterfactual). "만약 다른 조치를 취했다면 결과가 달랐을까?" $P(Y_x \mid X = x', Y = y')$ — 개체 수준 인과 효과(ITE)를 추정한다. 추천 시스템 맥락에서 "이 고객에게 A가 아닌 B를 추천했다면 어떤 결과였을까"에 해당한다.

상관관계의 함정과 교란 변수

추천 시스템에서 상관관계에 의존하면 교란 변수에 취약해진다. "프리미엄 카드 보유 $\leftarrow$ 높은 소득 $\rightarrow$ 여행 보험 가입"이라는 구조에서, 높은 소득이 두 변수 모두에 영향을 미치는 교란 변수이다. 상관관계 기반 시스템은 프리미엄 카드 보유자에게 여행 보험을 추천하지만, 카드를 무료 제공해도 보험 가입률은 변하지 않을 수 있다.

Causal Expert는 인접행렬 $\boldsymbol{W}$를 통해 변수 간 인과 방향과 강도를 학습하여, 개입 수준(제2단계)의 추론 구조를 추천에 내재시킨다. $W^2_{i,j}$는 "변수 j를 개입하여 변경하면 변수 i가 얼마나 변하는가"를 인코딩한다.

NOTEARS 연속 DAG 제약

DAG 구조 학습의 난제

비순환 방향 그래프(DAG) 구조 학습은 전통적으로 NP-hard 문제이다. 변수 d개에 대해 가능한 DAG 수는 초지수적으로 증가한다 ($d = 10$에서 약 4.2×10^{18}개). 기존의 제약 기반(PC 알고리즘) 또는 스코어 기반(GES) 접근법은 조합 탐색의 한계를 벗어나지 못했다.

NOTEARS 핵심 아이디어

Zheng et al. (NeurIPS 2018)의 NOTEARS (Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning)는 비순환성을 미분 가능한 연속 등식 제약으로 변환하여 조합 탐색을 완전히 우회한다.

$$h(\boldsymbol{W}) = \mathrm{tr}(e^{\boldsymbol{W} \circ \boldsymbol{W}}) - d = 0 \tag{1}$$

여기서:

- $e^{\boldsymbol{M}}$: 행렬 지수함수 (matrix exponential)
- $\mathrm{tr}(\cdot)$: 대각합 (trace)

- d: 인과 변수 수 (본 구현에서 $d = 32$)
- $\boldsymbol{W} \circ \boldsymbol{W}$: Hadamard (원소별) 제곱으로 비음수 인과 강도 보장

수학적 해석. 행렬 $\boldsymbol{A}$의 k-거듭제곱 $\left(\boldsymbol{A}^k\right)_{i,i}$은 노드 i에서 자기 자신으로 돌아오는 길이 k인 경로의 가중합이다. 따라서 행렬 지수 $e^{\boldsymbol{A}} = \sum_{k=0}^{\infty} \frac{\boldsymbol{A}^k}{k!}$의 대각 원소 $\left(e^{\boldsymbol{A}}\right)_{i,i}$는 노드 i에서 자기 자신으로 돌아오는 모든 길이의 순환 경로 가중합이다.

DAG에서는 순환 경로가 존재하지 않으므로, $k \geq 1$인 모든 항에서 대각 원소가 0이 된다. 오직 $k = 0$ (항등행렬)의 기여분 1만 남아 $e^{\boldsymbol{A}}_{i,i} = 1$이고, $\mathrm{tr}\left(e^{\boldsymbol{A}}\right) = d$이다. 따라서 $h(\boldsymbol{W}) = 0$은 그래프가 DAG임을 보장한다.

Taylor 10항 근사

행렬 지수함수를 직접 계산하면 고유값 분해 등 $O(d^3)$ 연산이 필요하다. 구현에서는 Taylor 급수의 $k = 1 \quad k = 10$ 항을 누적하여 근사한다.

$$e^{\boldsymbol{M}} = \boldsymbol{I} + \sum_{k=1}^{\infty} \frac{\boldsymbol{M}^k}{k!} \approx \boldsymbol{I} + \sum_{k=1}^{10} \frac{\boldsymbol{M}^k}{k!} \tag{2}$$

실제 구현에서는 $k = 0$ 항(항등행렬)을 루프에 포함하지 않고, $h(\boldsymbol{W}) = \mathrm{tr}\left(e^{\boldsymbol{M}}\right) - d$의 $-d$ 차감이 항등행렬 기여분을 제거한다. 루프는 $k = 1 \quad k = 10$을 누적한다:

```
M_power = torch.eye(d, device=W.device, dtype=W.dtype)
for i in range(1, 11):       # i = 1, 2, ..., 10  (k=1..10)
    M_power = M_power @ W_sq / i  # k!이 자동 구성됨
    h = h + torch.trace(M_power)  # tr(M^k / k!) 누적
# h 는 tr(e^M) - d 에 해당 (k=0 항등행렬의 trace = d 를 빼는 것과 동치)
```

10항이면 길이 10 이하인 모든 순환 경로를 감지하며, 32개 노드 DAG에서 10-hop 이상의 순환은 현실적으로 발생하기 어렵다. $\boldsymbol{W}$ 원소가 소규모 (0.01 초기화)일 때 고차항은 급격히 감소하므로 실용적 정밀도가 충분하다.

SCM (Structural Causal Model) 개입

Feature Compressor

고차원 피처를 DAG 변수 공간으로 압축하는 병목 구조이다.

$$\boldsymbol{z} = \mathrm{Compressor}(\boldsymbol{x}): \quad \mathbb{R}^{644} \to \mathbb{R}^{128} \to \mathbb{R}^{32} \tag{3}$$

구성: `Linear(644, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 32)`.

644개 원시 피처를 한꺼번에 인과 분석하면 변수 간 관계가 $644^2 \approx 41$만 개로 폭발한다. Compressor는 이를 32개 핵심 인과 변수로 요약하여 DAG 학습의 계산 부담을 해소한다.

인과 개입 연산

$$\hat{\boldsymbol{z}} = \boldsymbol{z} + \boldsymbol{z}(\boldsymbol{W} \circ \boldsymbol{W}) \tag{4}$$

여기서:
- $\boldsymbol{W} \in \mathbb{R}^{32 \times 32}$: 학습 가능한 가중 인접행렬 (`nn.Parameter`)
- $\boldsymbol{W} \circ \boldsymbol{W}$: Hadamard 제곱으로 비음수 인과 강도 보장
- $W^2_{i,j}$: 변수 $j \to$ 변수 i 방향의 인과 영향 강도
- 잔차 연결 ($\boldsymbol{z} +$): 원래 정보를 보존하면서 인과적 보정 추가

각 고객의 잠재 특성 벡터 $\boldsymbol{z}$를 "다른 특성들이 미치는 인과적 영향"만큼 보정한 버전 $\hat{\boldsymbol{z}}$를 생성한다. 이를 통해 단순 상관이 아닌 인과적으로 조정된 표현을 얻는다.

인접행렬 $\boldsymbol{W}$는 `torch.randn(32, 32) * 0.01`로 소규모 초기화되어 학습 초기 안정성을 보장한다.

Causal Encoder

인과 개입된 표현을 최종 출력 차원으로 변환한다.

$$\boldsymbol{o} = \text{CausalEncoder}(\hat{\boldsymbol{z}}): \quad \mathbb{R}^{32} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{5}$$

구성: `Linear(32, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 64)` → `LayerNorm(64)` → `SiLU`.

64D 출력은 다른 Expert(PersLay, DeepFM 등)와 동일 차원이어서 CGC Gate Attention에서 직접 비교 및 가중합이 가능하다.

DAG 정규화 손실

학습 과정에서 DAG 구조를 유지하기 위해 두 가지 정규화 항을 사용한다.

$$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \|\boldsymbol{W} \circ \boldsymbol{W}\|_1 \tag{6}$$

하이퍼파라미터	기본값	역할
`dag_lambda`	`0.01`	비순환성 제약 강도
`sparsity_lambda`	`0.001`	인접행렬 희소성 (L1)
`n_causal_vars`	`32`	인과 변수 수 = DAG 노드 수

비순환성 항 $h(\boldsymbol{W})$는 순환 금지를 강제하고, 희소성 항 $\|\boldsymbol{W} \circ \boldsymbol{W}\|_1 = \sum_{i,j} W_{i,j}^2$는 가능한 $32 \times 32 = 1024$개 간선 중 소수의 의미 있는 인과 관계만 남기도록 유도한다.

논문과의 차이. 원 논문은 $h(\boldsymbol{W}) = 0$ 등식 제약을 증강 라그랑지안(augmented Lagrangian)으로 엄격히 만족시키지만, 본 구현은 단순 페널티 $\lambda \cdot h(\boldsymbol{W})$로 완화하였다. MTL의 다른 손실 항과 균형을 유지하면서 근사적 비순환성을 달성하기 위한 실용적 선택이다. $\lambda = 0.01$에서 학습 완료 후 $h(\boldsymbol{W}) < 0.1$ 수준으로 수렴하면 충분하다.

> **주의:** `dag_lambda` > 0.1로 설정하면 인접행렬 $\boldsymbol{W}$가 영행렬로 수렴하여 인과 구조 자체가 소멸한다. DAG 정규화 손실이 태스크 손실보다 지배적이 되면 Expert가 항등 함수 ($\hat{\boldsymbol{z}} \approx \boldsymbol{z}$)로 퇴화한다.

금융 도메인 적용

행동 간 인과 방향 학습

금융 도메인에서 Causal Expert는 고객 행동 변수 간 인과 방향을 학습한다.

- **교란 변수 통제**: "카드 사용량과 충성도의 상관"에서 소득이라는 교란 변수를 통제하여, 카드 사용이 충성도에 미치는 순수 인과 효과만 추출한다.
- **개입 효과 예측**: "이 혜택을 제공하면 고객 행동이 어떻게 변하는가"에 대한 인과적 근거를 내재화한다.
- **설명력 강화**: `get_causal_graph()`로 인접행렬 $\boldsymbol{W} \circ \boldsymbol{W}$를 추출하여, 어떤 잠재 인과 변수가 다른 변수에 강하게 영향을 미치는지 히트맵으로 시각화할 수 있다.

인과 그래프 추출

```
def get_causal_graph(self) -> torch.Tensor:
    """graph[i,j] = W[i,j]^2: 변수 j -> 변수 i 인과 강도"""
    return (self.W * self.W).detach()
```

반환된 `[32, 32]` 행렬로 인과 경로 추적, 희소성 모니터링, 추천 사유 생성(LLM 그라운딩)에 활용한다.

논문 원문 vs 구현 비교

항목	논문 (Zheng et al.)	본 구현
목적	관측 데이터에서 DAG 구조 학습	Expert 내 피처 인과 관계 학습
변수 수	수십 수백 (범용)	32 고정
비순환성	증강 라그랑지안 (등식 제약)	단순 페널티: $\lambda \cdot h(\boldsymbol{W})$
인접행렬	$\boldsymbol{W}$ (부호 무관)	$\boldsymbol{W} \circ \boldsymbol{W}$ (비음수)
학습 방식	독립 최적화	End-to-end MTL 공동 학습
출력	DAG 인접행렬	64D 인과 표현 + DAG (시각화용)

Optimal Transport Expert

Wasserstein 거리 vs KL 발산

분포 비교의 기하학적 관점

두 확률 분포의 차이를 측정하는 전통적 방법은 다음과 같다:

- **KL 발산**: $D_{\mathrm{KL}}(P \parallel Q) = \sum_i P_i \log\left(\frac{P_i}{Q_i}\right)$
- **JS 발산**: KL 발산의 대칭 버전
- **총 변동 거리**: $\mathrm{TV}(P, Q) = \frac{1}{2}\sum_i |P_i - Q_i|$

이들은 공통적으로 분포의 기하학적 구조를 무시한다는 한계가 있다. 분포 P가 "서울"에 집중, Q가 "인천"에 집중, R이 "부산"에 집중되어 있을 때, KL 발산이나 TV 거리로는 $\mathrm{dist}(P, Q) \approx \mathrm{dist}(P, R)$이다 — 질량이 겹치지 않으면 거리가 동일하다. 그러나 직관적으로 서울-인천은 서울-부산보다 가깝다.

Wasserstein 거리의 장점

Wasserstein 거리 (earth mover's distance)는 기저 공간의 거리 구조를 반영한다.

$$W(\boldsymbol{\mu}, \boldsymbol{\nu}) = \min_{\boldsymbol{P} \in \mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C} \rangle_F \tag{7}$$

여기서:

- $\boldsymbol{\mu}, \boldsymbol{\nu} \in \Delta^d$: 소스와 타겟 분포 (확률 simplex 위의 벡터)
- $\boldsymbol{C} \in \mathbb{R}^{d \times d}$: 비용 행렬 — $C_{i,j}$는 위치 i에서 j로 질량 1단위를 수송하는 비용
- $\boldsymbol{P}$: 수송 계획 — $P_{i,j}$는 i에서 j로 실제 수송하는 질량
- $\mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu}) = \{\boldsymbol{P} \geq 0 : \boldsymbol{P}\mathbf{1} = \boldsymbol{\mu}, \boldsymbol{P}^\top \mathbf{1} = \boldsymbol{\nu}\}$: 주변 분포 제약

Wasserstein 거리가 KL 발산보다 우월한 핵심 이유:

속성	KL 발산	Wasserstein 거리
겹치지 않는 분포	$Q_i = 0$이면 정의 불가 (∞)	항상 유한 값
기하학적 인식	기저 공간 거리 무시	비용 행렬로 반영
연속적 변형	불연속적 변화 가능	분포 이동에 따라 연속 변화
수송 계획	스칼라 값만 제공	어떻게 변환하는지 ($\boldsymbol{P}$) 제공
대칭성	비대칭 ($D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$)	대칭 (거리 함수)

Sinkhorn 엔트로피 정규화

Kantorovich 문제의 계산적 한계

원래의 Kantorovich 최적 수송 문제는 d^2개 변수, $2d$개 등식 제약을 가진 선형계획(LP) 문제로, $O(d^3 \log d)$의 계산 비용이 발생한다. 실시간 추론에는 비실용적이다.

엔트로피 정규화

Cuturi (NeurIPS 2013)의 핵심 통찰: 엔트로피 항을 추가하면 문제의 구조가 근본적으로 변한다.

$$\min_{\boldsymbol{P}\in\mathcal{U}(\boldsymbol{\mu},\boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C}\rangle + \varepsilon \cdot H(\boldsymbol{P}) \tag{8}$$

여기서:

- $H(\boldsymbol{P}) = -\sum_{i,j} P_{i,j} \log P_{i,j}$: 수송 계획의 엔트로피
- $\varepsilon = 0.1$: 정규화 계수

엔트로피 항이 추가되면 강볼록(strictly convex) 문제가 되어 유일해가 존재하며, 최적해는 $\boldsymbol{P}^* = \mathrm{diag}(\boldsymbol{a})\boldsymbol{K}\ \mathrm{diag}(\boldsymbol{b})$ 형태를 가진다. 여기서 $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$는 Gibbs 커널이고, $\boldsymbol{a}, \boldsymbol{b}$는 Sinkhorn 교대 정규화로 구하는 스케일링 벡터이다.

계산 복잡도는 $O\left(\frac{d^2}{\varepsilon^2}\right)$로 감소하여 GPU 병렬화에 적합하다.

Sinkhorn 알고리즘의 직관

Sinkhorn 알고리즘은 교대 프로젝션(alternating projection)이다:

1. Gibbs 커널 $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$을 초기 수송 계획으로 삼는다
2. 행 정규화: 각 행의 합이 소스 분포 $\boldsymbol{\mu}$와 일치하도록 조정
3. 열 정규화: 각 열의 합이 타겟 분포 $\boldsymbol{\nu}$와 일치하도록 조정
4. 행/열 정규화를 교대 반복하면 양쪽 주변 분포 제약을 동시에 만족하는 최적 수송 계획에 수렴

Log-domain Sinkhorn 알고리즘

수치 안정성 문제

표준 Sinkhorn은 Gibbs 커널 $\boldsymbol{K} = \exp\left(-\frac{\boldsymbol{C}}{\varepsilon}\right)$의 원소를 직접 다루는데, ε이 작으면 $\exp\left(-\frac{C_{i,j}}{\varepsilon}\right)$이 극도로 작은 값이 되어 부동소수점 언더플로우가 발생한다.

Log-domain 쌍대 변수 업데이트

$$\boldsymbol{u}_{\text{new}} = \log \boldsymbol{\mu} - \mathrm{logsumexp}_j\left(-\frac{C_{i,j}}{\varepsilon} + v_j\right) \tag{9}$$

$$\boldsymbol{v}_{\text{new}} = \log \boldsymbol{\nu} - \mathrm{logsumexp}_i\left(-\frac{C_{i,j}}{\varepsilon} + u_i\right) \tag{10}$$

여기서:

- $\boldsymbol{u}, \boldsymbol{v}$: log-domain 쌍대 변수
- $\log \boldsymbol{K} = -\frac{\boldsymbol{C}}{\varepsilon}$: log-domain Gibbs 커널
- `logsumexp`: $\log \sum_j \exp(a_j)$ — 오버/언더플로우 방지

구현 참고: 이론적 정의에서 $\boldsymbol{v}$ 업데이트는 $\boldsymbol{C}^\top$을 사용하지만, 본 구현에서는 PSD 비용 행렬($\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M}$)이 대칭이므로 $\boldsymbol{C}^\top = \boldsymbol{C}$이다. 코드에서는 전치 대신 `logsumexp`의 축(dim)을 변경하여 동일한 결과를 얻는다.

- 반복 횟수: 10회 (기본값, `sinkhorn_iterations`)

수렴 후 수송 계획과 Wasserstein 거리를 계산한다:

$$\log \boldsymbol{P} = \boldsymbol{u} \oplus \log \boldsymbol{K} \oplus \boldsymbol{v}, \quad \text{즉} \quad \log P_{i,j} = u_i + \log K_{i,j} + v_j \tag{11}$$

$$W(\boldsymbol{\mu}, \boldsymbol{\nu}_k) = \langle \boldsymbol{P}, \boldsymbol{C}\rangle_F = \sum_{i,j} P_{i,j} \cdot C_{i,j} \tag{12}$$

학습 가능한 기준 분포와 PSD 비용 행렬

Distribution Projection

고객 피처를 확률 simplex로 변환한다.

$$\boldsymbol{\mu} = \mathrm{softmax}(\mathrm{DistProjector}(\boldsymbol{x})) \in \Delta^{32} \tag{13}$$

구성: `Linear(644, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 32)` → `softmax`.

softmax를 적용하면 32개 차원의 합이 1이 되어, 각 차원을 "소비 성향 카테고리에 배분하는 비율"로 해석할 수 있다.

학습 가능한 기준 분포

$$\boldsymbol{\nu}_k = \mathrm{softmax}(\ell_k) \in \Delta^{32}, \quad k = 1, ..., 16 \tag{14}$$

- ℓ_k: `nn.Parameter [16, 32]` — 학습 가능한 로짓
- 초기화: `torch.randn(16, 32) * 0.1`
- 각 $\boldsymbol{\nu}_k$는 전형적 고객 유형의 분포를 학습

16개 기준 분포는 사전 정의된 것이 아니라 학습 과정에서 데이터가 자연스럽게 군집화한 유형을 포착한다. 예를 들어 $\boldsymbol{\nu}_3$이 "여행 중심 소비형", $\boldsymbol{\nu}_7$이 "생활비 절약형"으로 학습될 수 있다.

PSD 비용 행렬

$$\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32 \times 32} \tag{15}$$

- $\boldsymbol{M}$: `nn.Parameter [32, 32]` — 학습 가능한 기저 행렬
- 초기화: $\boldsymbol{I} + \mathcal{N}(0, 0.01)$

PSD 보장: $\boldsymbol{x}^\top(\boldsymbol{M}^\top \boldsymbol{M})\boldsymbol{x} = \|\boldsymbol{M}\boldsymbol{x}\|^2 \geq 0$. 비용의 비음수성이 자동 보장되어 "수송할수록 이득"이라는 비물리적 상황을 원천 차단한다. 비용이 학습 가능하므로 태스크에 따라 의미론적 거리를 학습한다 — 예: "식비→외식은 저비용, 식비→여행은 고비용."

Wasserstein 거리 벡터

$$\boldsymbol{w} = [W(\boldsymbol{\mu}, \boldsymbol{\nu}_1), W(\boldsymbol{\mu}, \boldsymbol{\nu}_2), ..., W(\boldsymbol{\mu}, \boldsymbol{\nu}_{16})] \in \mathbb{R}^{16} \tag{16}$$

각 고객의 소비 분포 $\boldsymbol{\mu}$를 16개 전형적 고객 유형과 비교하여, 16차원 거리 벡터로 표현한다. 이것은 분포적 좌표계(distributional coordinate system)이다 — 고객을 16개 기준점으로부터의 거리로 위치시킨다.

Wasserstein Encoder

$$\boldsymbol{o} = \mathrm{WassersteinEncoder}(\boldsymbol{w}): \quad \mathbb{R}^{16} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{17}$$

구성: `Linear(16, 128)` → `LayerNorm(128)` → `SiLU` → `Dropout(0.2)` → `Linear(128, 64)` → `LayerNorm(64)` → `SiLU`.

금융 도메인 적용

세그먼트 간 분포 이동

금융 도메인에서 OT Expert는 고객의 소비 패턴 분포와 전형적 프로필 분포 사이의 구조적 매칭을 수행한다.

Wasserstein 거리는 "이 고객의 소비 패턴이 전형적인 여행형/저축형/외식형 프로필과 얼마나 다르며, 어떤 카테고리를 어느 방향으로 이동시키면 일치하는가"를 정량화한다. 수송 계획 P는 단순 스칼라 거리뿐 아니라, 구체적인 변환 방향을 제공한다.

논문 원문 vs 구현 비교

항목	논문 (Cuturi 2013)	본 구현
분포 입력	고정 이산/연속 분포	학습된 softmax 분포 (32D)
비용 행렬	고정 (유클리드 등)	학습 가능 PSD: $M^\top M$
기준 분포	단일 타겟 분포	16개 학습 가능 프로토타입
Sinkhorn 구현	표준 도메인 (행렬곱)	Log-domain (수치 안정성)
반복 횟수	수렴까지	고정 10회
출력	스칼라 OT 거리	16D 거리 벡터 → 64D 인코딩

FP16 수치 안정성

혼합 정밀도 (AMP) 환경의 도전

g4dn T4 GPU에서 AMP (Automatic Mixed Precision)를 활성화하면 학습 속도가 약 2배 향상되지만, FP16의 표현 범위 제한 ($\approx 6 \times 10^{-8} \quad 6.5 \times 10^4$)으로 인해 수치 안정성 문제가 발생한다.

Causal Expert FP16 이슈

Taylor 근사에서의 오버플로우

NOTEARS Taylor 10항 근사에서 $\boldsymbol{M}^k$ 누적 행렬곱은 k가 증가할수록 원소 크기가 급격히 증가할 수 있다. FP16에서 6.5×10^4를 초과하면 `inf`가 되고, `trace`에서 NaN으로 전파된다.

해결:

- $\boldsymbol{W}$ 초기화 스케일을 `0.01`로 유지하여 $\boldsymbol{W} \circ \boldsymbol{W}$ 원소가 10^{-4} 수준이 되도록 한다
- Gradient clipping (`gradient_clip_norm: 5.0`)으로 $\boldsymbol{W}$ 원소의 급격한 증가를 방지한다
- DAG 정규화 계산은 `torch.float32`로 캐스팅 후 수행하는 것을 권장한다

인접행렬 원소 폭주

`dag_lambda`가 너무 작으면 비순환성 제약이 약해져 $\boldsymbol{W}$ 원소가 커질 수 있고, 이는 Taylor 근사의 고차항에서 오버플로우를 유발한다.

OT Expert FP16 이슈

Sinkhorn log-domain에서의 문제

Log-domain Sinkhorn에서 `logsumexp` 연산은 FP16에서 다음 문제를 일으킬 수 있다:

1. **softmax 확률의 언더플로우**: `F.softmax(dist_logits, dim=-1)` 결과에서 매우 작은 확률 값이 FP16 최소값 이하로 내려가 0이 된다. 이후 `torch.log(a.clamp(min=1e-8))`에서 `1e-8`이 FP16에서 정확히 표현되지 않는다.
2. **비용 행렬 스케일**: $-\frac{C}{\varepsilon}$ 계산에서 $\varepsilon = 0.1$이면 비용 값이 10배 증폭된다. FP16 범위를 초과하면 NaN이 발생한다.

해결:

- `clamp(min=1e-7)` 대신 `clamp(min=1e-6)`로 변경. Sinkhorn 내부를 FP32로 캐스팅하므로 `1e-6`은 FP32에서 안전하게 표현된다.
- Sinkhorn 내부 연산을 `torch.float32`로 수행하고 결과만 FP16으로 캐스팅
- `@torch.cuda.amp.custom_fwd(cast_inputs=torch.float32)` 데코레이터 활용

Phase 2 NaN 수정 경험

실제 학습 중 발생한 NaN 이슈와 해결 과정:

Expert	증상	원인 및 해결
Causal	`causal_loss`가 epoch 3 이후 NaN	$\boldsymbol{W}$ 원소가 FP16 범위 초과. Taylor 고차항 오버플로우. → DAG 정규화를 FP32로 캐스팅, grad clip 5.0→1.0
OT	Sinkhorn 5회차 이후 NaN 전파	$-\frac{C}{\varepsilon}$에서 비용 스케일 증폭. → Sinkhorn 내부를 FP32로 수행, `cost_matrix` 초기화 $\boldsymbol{I} + \mathcal{N}(0, 0.01)$ 확인
OT	특정 배치에서만 간헐적 NaN	softmax 출력에 극단적 원-핫 분포 발생, `log(0)` 전파. → `clamp(min=1e-6)` 적용 (FP32 캐스팅으로 안전)

FP16 원칙: 두 Expert 모두 핵심 수치 연산 (Taylor 행렬 지수, Sinkhorn log-domain)은 FP32로 수행하고, 입출력 텐서만 FP16으로 캐스팅한다. AMP의 `GradScaler`가 NaN gradient를 감지하면 해당 step을 자동 스킵하지만, 빈번한 스킵은 학습 품질을 저하시키므로 근본 원인을 해결해야 한다.

구현 참고사항

두 Expert의 공통 설계 패턴

항목	Causal Expert	OT Expert
등록 이름	`"causal"`	`"optimal_transport"`
입력 차원	161D (demographics, products, txn, derived_temporal, product_hierarchy, gmm)	127D (demographics, products, txn, derived_temporal, gmm)
잠재 공간	32D (인과 변수)	32D (확률 simplex)
출력 차원	64D	64D
핵심 메커니즘	SCM: $\hat{z} = z + z(W \circ W)$	Sinkhorn: $W(\mu, \nu_k) \times 16$
학습 파라미터	W `[32,32]` 인접행렬	`reference_logits [16,32]` + `cost_matrix [32,32]`
정규화 손실	$\mathcal{L}_{\text{DAG}}$ (비순환성 + 희소성)	없음 (엔트로피 정규화 내재)
해석 출력	4D (인과 강도 등)	4D (수송 비용 등)

PLE 통합

두 Expert 모두 동일한 경로로 PLE에 통합된다:

```
# ple_cluster_adatt.py
elif name in ("causal", "optimal_transport"):
    out, _ = expert(inputs.features[:, expert_slice])  # FeatureRouter가 expert별 서브셋 슬라이싱
```

Causal Expert만 추가 정규화 손실을 가진다:

```
# ple_cluster_adatt.py -- compute_loss
if self.training and "causal" in self.shared_experts:
    dag_reg = self.shared_experts["causal"].get_dag_regularization()
    total_loss = total_loss + dag_reg
```

OT Expert는 별도 정규화 손실 없이 태스크 손실의 역전파를 통해 `reference_logits`와 `cost_matrix`가 자연스럽게 학습된다.

세 Expert의 수학적 관점 비교

DeepFM, Causal, OT 세 Expert는 각각 서로 다른 피처 서브셋을 입력받지만(Causal: 161D, OT: 127D, FeatureRouter를 통한 슬라이싱), 추출하는 수학적 구조는 근본적으로 다르다:

Expert	추출 대상	수학적 성질	고유 기여
DeepFM	피처 쌍 대칭 상호작용 $\langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle$	교환 가능 ($i \leftrightarrow j$)	2차 교차를 $O(nk)$에 포착
Causal	방향성 인과 $W_{i,j}^2$	비대칭, 비순환 (DAG)	교란 제거, 인과 방향
OT	분포 거리 $W(\boldsymbol{\mu}, \boldsymbol{\nu}_k)$	거리 함수 (metric)	분포의 기하학적 위치

Gradient Flow

Causal Expert

```
total_loss
  |-- task_loss -> causal_encoder -> _apply_causal_mechanism
  |                                     -> W (nn.Parameter) <- gradient
  |-- dag_loss -> get_dag_regularization()
                     -> W (nn.Parameter) <- gradient
```

$\boldsymbol{W}$는 태스크 손실과 DAG 정규화 손실 양쪽에서 gradient를 받는다. 두 gradient의 균형이 `dag_lambda` 설정으로 제어된다.

OT Expert

```
total_loss -> wasserstein_encoder -> _sinkhorn_distance -> dist_projector
                                        |-- cost_matrix <- gradient
                                        |-- reference_logits <- gradient
```

Sinkhorn의 10회 반복은 unrolled gradient를 사용한다. 반복 횟수가 많을수록 gradient 체인이 길어져 vanishing/exploding gradient 위험이 증가하므로, gradient clipping (`gradient_clip_norm: 5.0`)과 함께 사용한다.

디버깅 가이드

Causal Expert 이슈

증상	원인	조치
DAG 붕괴 ($\boldsymbol{W} \approx 0$)	`dag_lambda` 과대 (> 0.1)	`dag_lambda`를 0.01 이하로
비순환성 위반 ($h(\boldsymbol{W}) \gg 0$)	`dag_lambda` 과소 또는 학습률 과대	`dag_lambda` 점진적 증가, 학습률 감소
`causal_loss` NaN	Taylor 근사 발산, $\boldsymbol{W}$ 원소 과대	gradient clip 강화 ($5.0 \to 1.0$), FP32 캐스팅
Expert 출력이 상수	$\boldsymbol{W}$ 학습 정체	per-expert LR 설정, 초기화 확인

OT Expert 이슈

증상	원인	조치
Sinkhorn 발산 (NaN/Inf)	ε 과소 (< 0.01) 또는 비용 스케일 과대	$\varepsilon \geq 0.1$, 비용 초기화 확인

증상	원인	조치
퇴화 분포 (원-핫 softmax)	`dist_projector` 로짓 과대	temperature scaling 또는 dropout 증가
모든 거리 동일	기준 분포 붕괴 또는 비용 영행렬	`reference_logits` 다양성 모니터링
OT 거리 음수	비용 행렬 PSD 보장 실패	`cost_matrix.T @ cost_matrix` 확인

추가 자료

- **NOTEARS / DAGMA**: Zheng et al. NeurIPS 2018; Bello et al. ICML 2022.
- **Sinkhorn OT**: Cuturi NeurIPS 2013; Sinkhorn 1964; Villani 2009 (Optimal Transport: Old and New).
- **인과 추론 기초**: Pearl 2009 (Causality, 2판); Rubin 1974.